Pedestrian Data - CAA Requirements

# CAA Activity Report – Pedestrian Counts (Reporting Requirements)

**Owner:** Rahul Vivek Sirpur
**Last Updated:** 03 Dec 2025 (AET)
**Status:** In Review
**Audience:** CAA Stakeholders, Data Engineering

## Overview

This page defines how **pedestrian count data** is prepared and presented specifically for the **CAA Activity Report**.

**Key Outcomes:**

- Counts are **grouped by counter name** (no direction or entry/exit splits).
- **Zero-filled** hourly and daily series for completeness.
- **Missing flag (±3 hours)** to detect outage-like gaps and enable clean analysis.
- Hour-of-day **rolling averages**: **4-week** and **52-week**.
- **Source-standardized naming** for `venueid` and `countername`.
- Support view for **last sensor capture** to monitor data freshness.

> **Scope note:** This work supports **CAA reporting for the Activity Report**. It is **not** a comparison project against CAA Postgres; however, reconciliation findings may be documented if relevant.

## Business Rules

### Grouping & Aggregation

- **Group by:** `countername`, `day`, `hour`.
- **Exclude:** Direction and entry/exit type (even if present in source).

#### Aggregation Rule (Explained)

Raw pedestrian data often contains **multiple sub-records for the same hour** because sensors capture **directional flows** (e.g., Northbound vs Southbound) or **entry/exit types**.

If we keep these separate, the Activity Report would show **two or more rows for the same hour**, which is not what CAA needs. Instead, we **aggregate** them into **one row per hour per counter**, by **summing the counts**.

#### Example: Raw Data

| counter_name | hour | direction | count |
|---|---|---|---|
| Dockland Waterfront | 09:00 | North | 120 |
| Dockland Waterfront | 09:00 | South | 95 |

#### Aggregated for Reporting

| counter_name | hour | count |
|---|---|---|
| Dockland Waterfront | 09:00 | 215 |

### Why do we do this?

- The Activity Report requires **total pedestrian activity**, not directional splits.
- It simplifies reporting and ensures consistency across all counters.

#### Zero-Fill (Completeness)

- Produce a **complete hourly timeline** per counter for the reporting window.
- Any hour with no raw record must have `counts = 0`.
- If an entire **day** has no data for a counter, **counts for all 24 hours of that day must be set to 0** to maintain continuity.

#### Missing Flag (±3 Hours)

- `missing_flag = TRUE` when the **previous 3 hours** (H−1..H−3) **and** the **next 3 hours** (H+1..H+3) are all missing (i.e., zero after zero-fill and no raw presence).
- **Use in analysis:** Exclude `missing_flag = TRUE` hours from multi-day/hour aggregations to avoid skew.

#### Rolling Averages (Same Hour-of-Day)

- `avg_4week`: Average of counts at the **same local hour-of-day** across the **previous 28 days**.
- `avg_52week`: Average of counts at the **same local hour-of-day** across the **previous 364 days**.

#### Naming Standards

- Use **actual source names** for `countername` and `venueid`.

#### Timezone & DST

- Report `day` and `hour` in **local time (AET/AEDT)**.

---

### Reporting Schema

**Logical View Name:** `PedestrainCounts_hourly`

| Column Name | Type | Description |
|---|---|---|
| `venueid` | STRING/INT | Unique venue identifier (source-aligned). |
| `countername` | STRING | Standardized **source** counter name; grouped at this level. |
| `day` | DATE | Local calendar day. |
| `hour` | INT (0–23) | Local hour-of-day (DST-aware). |
| `count` | INT | Hourly count; **0** where data is missing (zero-fill). |
| `avg_4week` | DOUBLE | Rolling average for same hour across **previous 28 days** (excl. flagged). |
| `avg_52week` | DOUBLE | Rolling average for same hour across **previous 364 days** (excl. flagged). |
| `missing+/-3hr` | BOOLEAN | TRUE if previous **3 hours** and next **3 hours** are missing. |

---

### Data Freshness (Support View)

Maintain a **support/monitoring view** in Databricks to track the most recent sensor data ingestion. This view provides operational visibility for the **Activity Report pipeline** and helps with early detection of outages.

#### Purpose

- Ensure stakeholders can quickly verify when each sensor last reported data.

- Identify stale or offline sensors before they impact reporting.

View Details

- **timestamp**: The most recent raw event timestamp captured in **Databricks**.
- **SensorID**: Source-standardized sensor name.

### Ingestion Frequency

- **Current cadence:** Data is ingested **daily** into Databricks.
- **Granularity:** Each ingestion includes **1-minute resolution data** for pedestrian counts.